

Day One

Every company starts with an idea. Day One turns a single sentence into an autonomous AI company — a founding team of specialist agents that research, argue, negotiate, and build in front of you.

Document

Product Requirements Document

Timeline

Aug 3 – Aug 11 (8 days)

Team

6 builders

Version

1.0

Executive Summary

Day One is an autonomous multi-agent startup simulator. A user submits a single idea — "a platform that helps university students find internships" — and instead of receiving one AI-generated answer, they watch a company assemble itself in real time. A CEO agent opens the meeting. Research, Product, Design, Engineering, Finance, and Marketing agents join, debate tradeoffs, push back on each other, and converge on a real founding decision set. The output is a full startup package: positioning, MVP scope, architecture, brand, go-to-market, and an investor-ready pitch.

The differentiator is not the deliverables — plenty of tools generate a business plan. The differentiator is that the user *watches the disagreement happen*. A pricing model gets challenged by Finance. A feature gets cut by the CEO to protect the launch date. That visible negotiation is the demo.

Why judges will remember this: Most hackathon AI products are single-prompt wrappers with a nice UI. Day One's core mechanic — agents that read each other's output, disagree, and revise — is genuinely hard to fake with a scripted demo, which is exactly why it will stand out. We are recommending a scoped **6-agent boardroom** (not dozens of agents) so the team can make this real, not simulated, within 8 days.

What we are optimizing for

- **A visible thinking process**, not a spinner. The product's "loading state" is the product.
- **A believable disagreement** — at least one moment per run where an agent's proposal is rejected or revised by another agent, in view of the user.
- **A polished, premium UI** that reads as a funded startup's product, not a hackathon dashboard.
- **A realistic 8-day build** with 6 people — we've deliberately cut scope (fewer agents, no free-form chat, constrained output schema) so what ships actually works live, on stage, without a fallback video.

Product Vision

Every founder has stared at a blank page trying to turn "I have an idea" into a company. Day One compresses the first six weeks of a startup — market research, positioning, MVP scoping, architecture, brand, and pitch — into an 8-minute simulation you can watch happen.

Long-term, Day One is not a novelty generator. It's a new interface pattern for AI collaboration: instead of one model impersonating an expert, specialized agents with different incentives negotiate toward a decision, the same way real teams do. The startup use case is the hackathon-sized wedge into a much bigger idea — **AI organizations that produce decisions, not just text.**

What it is

An autonomous company simulation where specialized AI agents collaborate, negotiate, and produce a real startup plan a user can act on.

What it isn't

Not a business-plan template filler. Not a chatbot with departments in its system prompt.
Not a linear pipeline dressed up as "agents."

Problem Statement

Aspiring founders — students, indie hackers, first-time builders — face a cold-start problem: they have an idea but no team to pressure-test it against. Existing AI tools make this worse in one specific way: they say yes to everything. Ask ChatGPT to validate your startup idea and it will enthusiastically help you build a bad one, because a single model has no incentive to disagree with itself.

What's missing isn't more information generation — it's **structured disagreement**. Real founding teams work because a Finance person kills a feature Engineering wants, and a CEO breaks the tie. That friction produces better decisions. No current tool simulates that friction; they simulate agreement.

Why this matters for the hackathon specifically

- Judges have seen dozens of "AI generates a business plan" submissions. They have not seen agents override each other on stage.
- The problem is easy to state in one sentence to a judge ("AI tools always say yes — ours argues"), which matters enormously for a 3-minute pitch.

User Personas

PRIMARY

Hamza, 21

CS student with three half-built side project ideas and no way to know which one is worth six months of nights and weekends. Wants a fast, credible gut-check before committing.

Needs: a market reality check, a scoped MVP, and something he can show a friend that looks legitimate — not a wall of AI text.

SECONDARY

Ayesha, 27

Product manager moonlighting on a startup idea. Knows how to write a PRD herself but wants a fast first draft with a technical and financial gut-check she can then edit.

Needs: a defensible starting document — tech stack, roadmap, revenue model — that saves her the first draft.

DEMO AUDIENCE

Hackathon Judge

Has watched 40 demos this week. Evaluates on originality, technical depth, and whether the "AI" part is real or window dressing.

Needs: proof, within 60 seconds, that this is not a single prompt in a nice wrapper.

TERTIARY

Early-stage Investor Scout

Could use Day One's output format (investor summary, elevator pitch, positioning) as a fast-triage tool when reviewing many incoming ideas.

Needs: a consistent, skimmable summary format across many different startup ideas.

User Journey

End-to-end flow

1. **Landing** — user sees the concept in one line and a single input field: "What's your idea?"
2. **Idea submission** — user types one sentence, optionally answers 2 quick clarifying chips (target market, B2B/B2C) if the idea is ambiguous.
3. **Opening the office** — cinematic loading sequence; CEO agent "joins," then each department joins in sequence with a short animated status line.
4. **Live boardroom** — the core screen. A structured feed shows agents posting findings, challenging each other, and the CEO resolving conflicts. Runs roughly 60–90 seconds.
5. **Resolution moment** — at least one visible "the CEO overruled X" or "Finance forced a scope cut" beat, called out visually.
6. **Company dossier** — final structured report renders: positioning, MVP, architecture, brand, roadmap, pitch. Organized as a tabbed/scrollable document, not a wall of markdown.
7. **Export / share** — user can copy the elevator pitch, download the dossier, or start a new company.

The entire journey from "idea submitted" to "dossier ready" should complete in under two minutes — long enough to feel substantial, short enough to hold a live demo audience.

Core Features

Feature	Description	Priority
Idea intake	Single free-text input, lightweight disambiguation chips	Must
Live boardroom feed	Real-time streamed agent messages with role identity, shown as a structured conversation, not raw chat bubbles	Must
Agent disagreement engine	At least one structured conflict-and-resolution cycle per run (see Section 7)	Must
Cinematic status narration	"Opening the office," "Research is analyzing competitors," etc., instead of generic spinners	Must
Structured company dossier	Final rendered output across all deliverables in Section 15	Must
Session replay	User can scroll back through the boardroom transcript after completion	Should
Export dossier (PDF/share link)	Downloadable or shareable version of the final report	Should
Multiple concurrent company runs	User can run more than one idea in the same session	Nice
User can "step in" mid-meeting	Interject with a constraint ("budget is \$500") that agents must respect	Nice
Pitch deck / landing page generation	Auto-generated slide deck or landing page copy from the dossier	Stretch

Multi-Agent Architecture

Scope call: the PRD brief asks us to avoid dozens of agents and recommend the number with the best impact-to-effort ratio. We recommend **6 agents** — one per team member, which also cleanly maps to the work breakdown in Section 23.

Recommended agent roster

- 1. CEO (Orchestrator)** — owns the meeting, assigns work, resolves conflicts, makes the final tradeoff call. The only agent allowed to overrule another agent's output.
- 2. Market & Research** — competitive landscape, target audience, market sizing signal, UVP input.
- 3. Product & Engineering** — MVP scope, roadmap, tech stack, system + database architecture. (Merged from the brief's separate Product/Engineering departments — see rationale below.)
- 4. Design** — brand direction, logo concept, UX principles for the MVP.
- 5. Finance & Legal** — cost reality check, revenue model, pricing sustainability, lightweight legal/compliance flags. (Merged — see rationale below.)
- 6. Marketing & Investor Relations** — go-to-market, launch checklist, investor summary, elevator pitch. (Merged.)

Why we merged three department pairs

The brief lists 10 departments. Ten independently-reasoning agents in 8 days is a coordination and prompt-engineering tax the team can't afford, and it dilutes the "wow" — more agents talking past each other reads as noisier, not smarter. We merged along natural tension lines that *preserve* the disagreement mechanic: Product+Engineering keeps the "is this buildable" tension internal to one agent's reasoning (still surfaced as a visible tradeoff line), while Finance+Legal and Marketing+Investor Relations combine roles that share the same audience and rarely conflict with each other. The tensions that matter for the demo — Research vs. Finance, Design vs. Engineering, Marketing vs. CEO — are all preserved.

Orchestration pattern: Blackboard + Orchestrator hybrid

We recommend a **shared-state ("blackboard") architecture with a CEO orchestrator**, not a fully decentralized agent-to-agent chat. This is the single most important technical decision in this PRD.

Pattern	Pros	Cons	Verdict
Fully decentralized (agents freely message each other)	Most "alive" in theory	Unpredictable runtime, hard to demo reliably, token cost explodes, risk of loops	Too risky for a timed live demo
Strict linear pipeline (agent 1 → 2 → 3...)	Simple, cheap, reliable	No real disagreement — exactly what the brief tells us to avoid	Fails the core requirement

Orchestrated blackboard (chosen)	CEO controls turn order and forces at least one conflict cycle; all agents read/write one shared state object; deterministic enough to demo live	Requires careful prompt design for the CEO's orchestration logic	Recommended
---	--	--	--------------------

How it works

1. CEO agent receives the idea, writes an initial brief to the shared **Company State** object, and assigns Research + Product/Eng to run in parallel (round 1).
2. Research and Product/Eng each read the brief, do their reasoning (with tool calls where relevant — e.g. web search for competitors), and write structured findings back to Company State.
3. CEO reads both outputs, and explicitly routes at least one to Finance and one to Design for challenge (round 2) — this is the forced-conflict step, not left to chance.
4. Finance/Design post a challenge object (e.g. `{type: "pricing_conflict", target: "product_eng", reason: "..."}`) to Company State.
5. CEO reads the challenge, decides: accept, ask target agent to revise, or overrule. This decision is what the "wow moment" UI beat is built from.
6. Marketing/Investor agent runs last, consuming the finalized state to produce GTM, pitch, and investor summary.
7. CEO compiles the final Company State into the rendered dossier schema.

Agent Responsibilities

Agent	Inputs	Outputs	Can challenge
CEO	Raw idea, all agent outputs	Brief, task assignments, conflict resolutions, final dossier compilation	Everyone
Market & Research	CEO brief	Problem statement, target audience, competitor analysis, UVP	—
Product & Engineering	CEO brief, Research output	MVP definition, roadmap, tech stack, system + DB architecture	Research (feasibility)
Design	Research + Product output	Brand name options, positioning, logo direction, UX principles	Product (usability)
Finance & Legal	Product + Design output	Revenue model, cost reality check, pricing sustainability, compliance flags	Product (cost), Marketing (pricing)
Marketing & Investor	Finalized state	GTM strategy, launch checklist, investor summary, elevator pitch	—

Agent Communication Flow

All agents communicate exclusively through the shared **Company State** object — never directly to each other. This is a deliberate constraint: it keeps the system observable (the UI just renders state diffs), debuggable (one object to inspect), and demo-safe (no risk of an unscripted agent-to-agent loop burning the live demo's time budget).

Example exchange (rendered live in the UI)

Research: "The internship-matching space is crowded — Handshake and LinkedIn dominate. Whitespace exists in *verified, university-specific* matching."

Marketing: "Gen Z demand for niche, trusted platforms over generic ones is growing — this supports a narrow wedge strategy."

Finance: "Proposed freemium-to-\$15/mo model won't clear CAC for a student audience. Recommend university B2B licensing instead."

Product/Engineering: "A recommendation-matching engine plus verification flow will delay MVP by 3+ weeks."

CEO: "Compromise: cut the matching engine for MVP, ship manual-curated listings + verification only. Revisit matching post-launch. Approved: university B2B pricing over consumer freemium."

Communication rules (enforced by schema, not prompting alone)

- Every agent output is a typed JSON object validated against a schema before being written to Company State — this prevents free-form drift that would break the render.
- A "challenge" is a first-class object type, not inferred from free text — this is what makes the conflict UI reliable to render.
- Only the CEO agent may write a `resolution` object; no other agent's output is treated as final.

Information Flow

The system is a single directed run per session — no persistence of company state between users' sessions is required for MVP (see Non-Functional Requirements).

1. **Client** submits idea → REST call creates a `company_run` record.
2. **Orchestration server** spins up the agent run as an async job, opens a WebSocket channel keyed to the run ID.
3. Each agent step's output is pushed over the WebSocket as a structured event (`agent_joined` , `agent_posted` , `challenge_raised` , `resolution_made` , `run_complete`).
4. **Client** renders each event incrementally into the boardroom feed as it arrives — no polling.
5. On `run_complete` , the client fetches the compiled dossier via REST and renders the final report view.

UX Flow

Screen	Purpose	Key interaction
Landing	One-line pitch + idea input	Single text field, subtle animated background suggesting "a company waiting to be born"
Opening sequence	Build anticipation, disguise cold-start latency	Sequential agent "joined" cards, ~5–8 seconds total
Live boardroom	The core demo moment	Auto-scrolling structured feed; challenge/resolution events visually distinct (color + icon) from normal posts
Company dossier	Deliverable consumption	Tabbed sections (Strategy / Product / Tech / Brand / Go-to-Market / Pitch), each independently scrollable
Export/share	Take the artifact away	Copy elevator pitch button, download button

UI Pages

- `/` — Landing + idea intake
- `/company/[runId]` — Live boardroom (streaming view)
- `/company/[runId]/dossier` — Final structured report
- `/company/[runId]/replay` — Scrollable transcript (Should Have)

Visual language: dark-on-light neutral base (near-black text, off-white background), single confident accent color (indigo/violet), generous whitespace, Inter or similar geometric sans typeface, no gradients-as-decoration, no stock "robot" iconography — this is what separates Linear/Vercel-tier design from a typical hackathon dashboard.

Functional Requirements

ID	Requirement	Priority
FR-1	User can submit a free-text startup idea and start a run	Must
FR-2	System streams agent events in real time via WebSocket	Must
FR-3	System produces at least one visible challenge + CEO resolution per run	Must
FR-4	System compiles a structured dossier covering all Section 15 deliverables	Must
FR-5	User can view a completed run's full transcript after the fact	Should
FR-6	User can export the dossier as PDF or shareable link	Should
FR-7	User can add a mid-run constraint that agents must incorporate	Nice
FR-8	System generates a slide deck from the dossier	Stretch

Non-Functional Requirements

- **Latency:** full run completes in under 2 minutes end-to-end (target 60–90s) — critical for a live demo, not just a "nice UX."
- **Reliability:** the run must not silently hang; every agent step has a timeout with a graceful fallback message.
- **Determinism-adjacent behavior:** the forced-conflict step (Section 7) guarantees at least one visible disagreement regardless of the specific idea submitted — this cannot be left to chance for a demo.
- **Cost control:** cap total tokens per run; use a smaller/faster model for research/formatting steps and reserve the strongest model for the CEO's resolution reasoning.
- **No auth required for MVP** — anonymous, session-based runs only; reduces build surface area.
- **Data retention:** runs persisted for the session/demo window only; no requirement for long-term user accounts in MVP.

MVP Scope

Deliverables the dossier must produce

Deliverable	Priority
Startup summary	Must
Problem statement	Must
Target audience	Must
Competitor analysis	Must
Unique Value Proposition	Must
MVP definition	Must
Product roadmap	Must
Recommended tech stack	Must
Database architecture (high level)	Should
System architecture (high level)	Should
Brand name suggestions	Must
Brand positioning	Must
Logo direction (text description, not generated image)	Should
Marketing strategy	Must
Launch checklist	Should
Revenue model	Must
Investor summary	Must
Elevator pitch	Must

Cut for realism: actual generated logo images, a real payment/investor CRM integration, and multi-round (3+) negotiation cycles are explicitly out of MVP scope. One well-executed conflict cycle beats three shallow ones, and it's what's actually achievable at demo-reliability in 8 days.

Stretch Features

- **Stretch** Landing page copy generated and rendered as an actual mini live preview
- **Stretch** Wireframe generation (low-fidelity, schema-driven boxes, not pixel-accurate)
- **Stretch** Social media launch campaign (3–5 sample posts)
- **Stretch** Auto-generated pitch deck (slides.pptx or rendered HTML deck)
- **Stretch** Risk analysis section with likelihood/impact scoring
- **Stretch** User "steps into the boardroom" and injects a constraint mid-run

Recommendation: attempt landing page copy + a live-rendered mini landing page first if time remains after MVP — it's the single highest visual-impact stretch goal per hour invested, since it reuses the dossier's brand + copy data with no new agent logic.

Technical Architecture

System diagram (described)

Client (Next.js) ⇌ WebSocket Gateway ⇌ Orchestration Service (FastAPI + async workers) ⇌ Agent Runner (LLM calls, tool use) ⇌ Company State Store (Postgres + Redis pub/sub) .
 Web search tool calls (for the Research agent) go through a single shared search API to control cost and rate limits.

Key architectural decisions

- **Orchestration framework:** LangGraph (or a hand-rolled state machine if the team prefers full control — see risk note below) to model the CEO's routing logic as an explicit graph rather than implicit prompt chaining.
- **Streaming:** WebSocket over polling — polling would either feel laggy or hammer the server; the "alive" feeling depends on true push updates.
- **State store:** Postgres for the durable Company State record (so a dossier can be reloaded/shared), Redis pub/sub purely for fanning out live events to connected clients.
- **Schema-first agent outputs:** every agent call uses structured output / tool-call-style JSON schemas, not free text parsed with regex — this is what makes the UI reliable enough to demo live.

Build-vs-frameworks risk: if any team member is not already comfortable with LangGraph (or CrewAI/ AutoGen), do not learn it during the hackathon. A hand-rolled orchestrator (a Python state machine with 6 explicit function calls and one shared dict) is less impressive on a resume but far more reliable under demo pressure, and judges cannot tell the difference from the outside.

Recommended Tech Stack

Layer	Recommendation	Why
Frontend	Next.js + TypeScript + Tailwind	Fast to build a premium-feeling UI; strong component ecosystem for streaming UIs
Realtime	WebSocket (native or Socket.IO)	Push-based event streaming for the boardroom feed
Backend / Orchestration	Python + FastAPI	Best-supported ecosystem for LLM orchestration tooling and async workers
Agent framework	LangGraph, or a hand-rolled state machine (see risk note above)	Explicit control flow over implicit agent chat
LLM provider	Claude (Sonnet-tier) via Anthropic API, structured outputs	Strong instruction following for schema-constrained agent outputs and multi-step reasoning
Database	PostgreSQL (hosted, e.g. Supabase/ Neon)	Durable Company State + dossier storage; free tier sufficient for hackathon scale
Pub/Sub	Redis	Fan-out of live agent events to connected clients
Search tool	A single web-search API (e.g. Tavily/ Serper)	Gives the Research agent real, current competitor data instead of hallucinated names
Hosting	Vercel (frontend) + Render/Railway (backend + Redis + Postgres)	Fastest path to a public demo URL with minimal DevOps overhead

Database Design

Core tables (high level)

Table	Key fields	Purpose
<code>company_runs</code>	id, idea_text, status, created_at, completed_at	One row per user session/run
<code>agent_events</code>	id, run_id, agent_name, event_type, payload (jsonb), sequence, created_at	Append-only log of every agent action — powers both live streaming and replay
<code>challenges</code>	id, run_id, raised_by, target_agent, reason, resolution, resolved_by	First-class conflict records — drives the "wow moment" UI
<code>dossiers</code>	id, run_id, section_key, content (jsonb)	Final compiled deliverable content, keyed by section for easy tabbed rendering

`agent_events` being append-only and jsonb-typed is the key decision: it means the frontend can be built entirely against one predictable event stream, and replay (Section 6, Should Have) is "free" — it's just re-reading the same table.

APIs and Integrations

Endpoint	Method	Purpose
/api/runs	POST	Create a new company run from an idea
/api/runs/:id	GET	Fetch run status and metadata
/api/runs/:id/dossier	GET	Fetch the compiled final dossier
/api/runs/:id/events	GET	Fetch full event transcript (for replay)
ws://.../runs/:id/stream	WS	Live event stream for an in-progress run

External integrations: Anthropic API (agent reasoning), one web search API (Research agent grounding). No payment, auth, or CRM integrations required for MVP — deliberately, to protect build time.

Risks and Mitigation

Risk	Impact	Mitigation
Live demo hits an LLM timeout or malformed output	High — breaks the pitch	Strict per-agent timeouts with fallback canned-but-idea-specific content; schema validation with retry-once logic; rehearse on a pre-tested idea as backup
Agent "disagreement" reads as fake/scripted	High — undermines the core pitch	Ground challenges in the actual Finance/Research numbers produced that run (not templated text), so wording varies run to run while structure stays reliable
Team underestimates orchestration complexity	High — could blow the whole timeline	Use the hand-rolled state machine fallback (Section 17) if LangGraph isn't already familiar to the team by Day 2
Token/API cost overrun during testing	Medium	Cap tokens per agent call, use a cheaper model for Research/formatting, set a hard budget alert
UI looks "hackathon-y" under time pressure	Medium — hurts judge perception	Lock the design system (Section 12) on Day 1–2 before feature work starts; reuse one component library throughout
Scope creep toward 10 agents	Medium	This PRD fixes the roster at 6 — treat any 7th agent proposal as a stretch-goal-only discussion

Development Roadmap — 8-Day Hackathon

Day	Focus	Exit criteria
Day 1 (Aug 3)	Architecture lock: finalize agent roster, schemas, DB models, design system tokens	Schemas + tech stack committed; no more architecture debate after today
Day 2	Backend skeleton: orchestration service, WebSocket gateway, DB tables, one agent (CEO) working end-to-end with dummy data	A hardcoded idea produces a fake but fully-streamed event sequence
Day 3	Frontend skeleton: landing page, live boardroom feed rendering the dummy stream	Full click-through demo works with fake data
Day 4	Wire in real agents: Research + Product/Engineering with real LLM calls	Real idea → real Research + Product output visible in UI
Day 5	Wire in Design, Finance/Legal, Marketing/Investor; implement the forced-conflict step	Full 6-agent run completes with at least one real challenge/resolution
Day 6	Dossier compilation + final report UI; error handling and timeouts	End-to-end run: idea in → full dossier rendered out, no manual intervention
Day 7	Polish pass: animations, design system consistency, copywriting pass on all UI text; attempt one stretch goal if ahead of schedule	Product looks and feels premium; run is demo-stable across 5+ consecutive tries
Day 8 (Aug 11)	Demo rehearsal, backup recording, pitch deck, submission	Live demo rehearsed 3+ times; fallback video recorded in case of live failure

Work Breakdown for 6 Team Members

Role	Owns	Days 1–3	Days 4–6	Days 7–8
Backend/ Orchestration Lead	Orchestration service, CEO agent logic, conflict resolution flow	State machine + schemas	Wire CEO routing + resolution logic	Reliability hardening
Agent Engineer (Research + Finance/ Legal)	Two agent prompt/tool pipelines	Prompt scaffolding, search tool integration	Live agent wiring, output validation	Prompt polish for demo idea
Agent Engineer (Product/Eng + Design + Marketing)	Three agent prompt/tool pipelines	Prompt scaffolding	Live agent wiring, output validation	Prompt polish
Frontend Lead	Boardroom feed, streaming render, design system	Component library, static boardroom mock	Wire to real WebSocket stream	Animation/motion polish
Frontend Engineer	Dossier view, landing page, export/replay	Landing + dossier static layout	Wire dossier to real data	Copywriting + polish
PM / Infra / Demo Lead	DB schema ownership, deployment, demo script, pitch narrative	DB migrations, hosting setup	Integration testing across the pipeline	Demo rehearsal, backup recording, submission

Demo Script (3–5 minutes)

1. **0:00–0:20 — Hook.** "Every AI tool says yes to your bad startup idea. Ours argues with you." Open Day One live.
2. **0:20–0:40 — Idea submission.** Type a real, slightly risky idea live (not the rehearsed safe one) to prove it's not scripted — or state clearly it's the tested idea if reliability requires it.
3. **0:40–1:10 — Opening the office.** Let the cinematic join sequence play; narrate what's happening rather than sitting in silence.
4. **1:10–2:30 — The boardroom.** Let Research and Product post first, then narrate the forced conflict as it appears: "Watch — Finance is about to kill this pricing model." Pause on the CEO's resolution line. This is the emotional peak of the demo.
5. **2:30–3:15 — The dossier.** Jump to the completed report; scroll through 3–4 sections fast (tech stack, brand, investor summary) to show depth without reading everything aloud.
6. **3:15–3:45 — The close.** Read the auto-generated elevator pitch aloud verbatim — it's a strong, concrete closing line that proves the system produced something genuinely usable.
7. **3:45–4:00 — The ask/vision.** One sentence on where this goes beyond the hackathon (Section 26).

Judge "Wow Moments"

- **The overrule moment** — seeing the CEO agent explicitly reject or revise another agent's proposal, in a UI treatment distinct enough that a judge notices it without narration.
- **Speed with substance** — a full, genuinely useful startup dossier in under two minutes, not a 10-minute wait for a shallow result.
- **Design credibility** — the product simply not looking like a hackathon project. This lowers judges' skepticism before they've evaluated a single feature.
- **The elevator pitch payoff** — ending the demo by reading a genuinely sharp, idea-specific one-liner the system produced, proving the output is usable, not filler.
- **Re-runability** — if time allows, running a second, judge-suggested idea live and getting a coherent, different-in-substance result reinforces this isn't a canned path.

Future Vision Beyond the Hackathon

Post-hackathon, Day One's roadmap has two directions worth pursuing independently:

1. Deeper startup co-founder product

Persistent companies (not one-shot runs), agents that revisit decisions as the user provides real traction data, integrations with real tools (Stripe for revenue modeling against actual numbers, GitHub for engineering agent to track real repo progress), and a genuine second negotiation round where the user can push back on the AI board, not just receive its output.

2. The underlying platform: AI organizations as a primitive

The blackboard-plus-orchestrator pattern built here — typed shared state, forced structured conflict, an accountable orchestrator agent — generalizes far past startups: legal review boards, product spec review committees, editorial rooms. The startup simulator is the most demoable, most universally-understood wedge into a broader "AI organizations" platform play, which is the more durable long-term thesis.

"Every company starts with an idea. The best ones survive an argument first."